

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

I G Vishnu Madhav (192425192)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

G Vishnu Madhav

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

ANNEXURE

Coding Challenge

1. Library Book Management using Classes & Encapsulation

```
import java.util.*;

class Book {

    private int id;

    private String title;

    private boolean issued;

    public Book(int id, String title) {

        this.id = id;

        this.title = title;

        issued = false;

    }

    public int getId() { return id; }

    public String getTitle() { return title; }

    public void issue() {

        if (!issued) {

            issued = true;

            System.out.println("Issued");

        } else {

            System.out.println("Already Issued");

        }

    }

    public void ret() {

        if (issued) {

            issued = false;

            System.out.println("Returned");

        }

    }

}
```

```
    } else {  
  
        System.out.println("Not Issued");  
  
    }
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Scanner s = new Scanner(System.in);  
  
        Book[] b = new Book[10];  
  
        int n = 0;  
  
        while (true) {  
  
            System.out.println("1.Add 2.Show 3.Issue 4.Return 5.Search 6.Exit");  
  
            int ch = s.nextInt();  
  
            if (ch == 1) {  
  
                System.out.print("ID: ");  
  
                int id = s.nextInt();  
  
                s.nextLine();  
  
                System.out.print("Title: ");  
  
                String t = s.nextLine();  
  
                b[n++] = new Book(id, t);  
  
            }  
  
            else if (ch == 2) {  
  
                for (int i = 0; i < n; i++) b[i].show();  
  
            }  
  
            else if (ch == 3) {  
  
                System.out.print("ID: ");  
  
                int id = s.nextInt();  
  
                for (int i = 0; i < n; i++)  
  
                    if (b[i].getId() == id) b[i].issue();  
  
            }  
  
            else if (ch == 4) {
```

```

        System.out.print("ID: ");

        int id = s.nextInt();

        for (int i = 0; i < n; i++)

            if (b[i].getId() == id) b[i].ret();

    }

    else if (ch == 5) {

        s.nextLine();

        System.out.print("Title: ");

        String t = s.nextLine();

        for (int i = 0; i < n; i++)

            if (b[i].getTitle().equalsIgnoreCase(t)) b[i].show();

    }

    else break;

}

}

}

```

The screenshot shows an IDE with a Java file named 'Main.java'. The code is a menu-driven program for a library system. It has a menu with options: 1.Add, 2.Show, 3.Issue, 4.Return, 5.Search, 6.Exit. The code implements options 1, 2, 3, 4, and 5. Option 1 adds a new book, option 2 shows all books, option 3 issues a book, option 4 returns a book, and option 5 searches for a book by title. The code uses an array of Book objects and a Scanner for input.

```

49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88

```

```

if (ch == 1) {
    System.out.print("ID: ");
    int id = s.nextInt();
    s.nextLine();
    System.out.print("Title: ");
    String t = s.nextLine();
    b[n++] = new Book(id, t);
}

else if (ch == 2) {
    for (int i = 0; i < n; i++) b[i].show();
}

else if (ch == 3) {
    System.out.print("ID: ");
    int id = s.nextInt();
    for (int i = 0; i < n; i++)
        if (b[i].getId() == id) b[i].issue();
}

else if (ch == 4) {
    System.out.print("ID: ");
    int id = s.nextInt();
    for (int i = 0; i < n; i++)
        if (b[i].getId() == id) b[i].ret();
}

else if (ch == 5) {
    s.nextLine();
    System.out.print("Title: ");
    String t = s.nextLine();
    for (int i = 0; i < n; i++)
        if (b[i].getTitle().equalsIgnoreCase(t)) b[i].show();
}

else break;
}
}
}

```

Input

```

1.Add 2.Show 3.Issue 4.Return 5.Search 6.Exit
1
ID: 154
Title: java
1.Add 2.Show 3.Issue 4.Return 5.Search 6.Exit
2
154 java Available
1.Add 2.Show 3.Issue 4.Return 5.Search 6.Exit
6
...Program finished with exit code 0
Press ENTER to exit console.

```

2. Employee Payroll System using Inheritance

```

class Employee {

```

```

    int id;

```

String name;

Employee(int id, String name) {

this.id = id;

this.name = name;

}

double calculateSalary() {

return 0;

}

void display() {

System.out.println(id + " " + name + " Salary: " + calculateSalary());

}

}

class PermanentEmployee extends Employee {

double basic, bonus;

PermanentEmployee(int id, String name, double basic, double bonus) {

super(id, name);

this.basic = basic;

this.bonus = bonus;

}

double calculateSalary() {

return basic + bonus;

}

}

class ContractEmployee extends Employee {

double hours, rate;

ContractEmployee(int id, String name, double hours, double rate) {

super(id, name);

this.hours = hours;

this.rate = rate;

```

    }

    double calculateSalary() {

        return hours * rate;

    }

}

public class Main {

    public static void main(String[] args) {

        Employee e;

        e = new PermanentEmployee(1, "Vishnu", 50000, 5000);

        e.display();

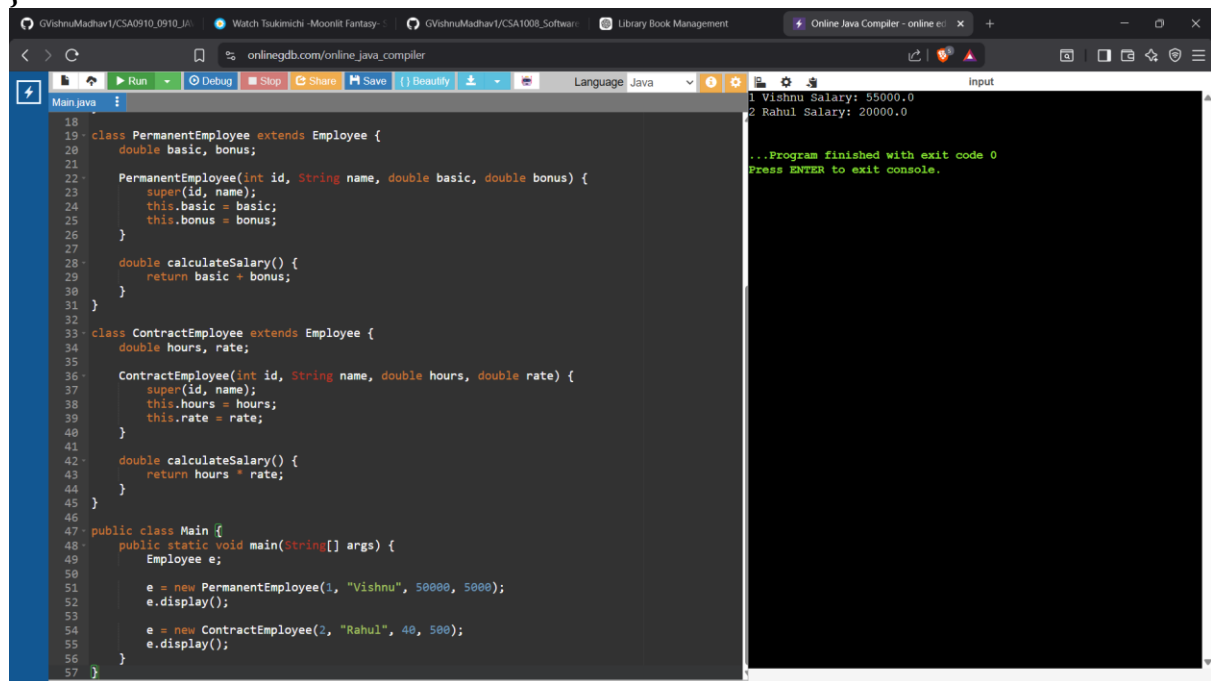
        e = new ContractEmployee(2, "Rahul", 40, 500);

        e.display();

    }

}

```



The screenshot shows an online Java compiler interface. The left pane displays the source code, which is a Java program demonstrating runtime polymorphism with two subclasses, `PermanentEmployee` and `ContractEmployee`, both extending a base class `Employee`. The `PermanentEmployee` class has attributes `basic` and `bonus`, while `ContractEmployee` has `hours` and `rate`. Both have a `calculateSalary()` method. The `Main` class creates instances of both and calls their `display()` methods. The right pane shows the output of the program, which prints the salary for Vishnu (55000.0) and Rahul (20000.0), followed by a message indicating the program finished successfully.

```

Main.java
18
19 class PermanentEmployee extends Employee {
20     double basic, bonus;
21
22     PermanentEmployee(int id, String name, double basic, double bonus) {
23         super(id, name);
24         this.basic = basic;
25         this.bonus = bonus;
26     }
27
28     double calculateSalary() {
29         return basic + bonus;
30     }
31 }
32
33 class ContractEmployee extends Employee {
34     double hours, rate;
35
36     ContractEmployee(int id, String name, double hours, double rate) {
37         super(id, name);
38         this.hours = hours;
39         this.rate = rate;
40     }
41
42     double calculateSalary() {
43         return hours * rate;
44     }
45 }
46
47 public class Main {
48     public static void main(String[] args) {
49         Employee e;
50
51         e = new PermanentEmployee(1, "Vishnu", 50000, 5000);
52         e.display();
53
54         e = new ContractEmployee(2, "Rahul", 40, 500);
55         e.display();
56     }
57 }

```

```

Vishnu Salary: 55000.0
Rahul Salary: 20000.0
...Program finished with exit code 0
Press ENTER to exit console.

```

3. Vehicle Rental System using Runtime Polymorphism

```

class Vehicle {

    String name;

    Vehicle(String name) {

        this.name = name;

    }

}

```

```
double calculateRent(int days) {  
    return 0;  
}  
  
void display(int days) {  
    System.out.println(name + " Rent: " + calculateRent(days));  
}  
  
class Car extends Vehicle {  
    Car(String name) {  
        super(name);  
    }  
  
    double calculateRent(int days) {  
        return days * 1000;  
    }  
}  
  
class Bike extends Vehicle {  
    Bike(String name) {  
        super(name);  
    }  
  
    double calculateRent(int days) {  
        return days * 300;  
    }  
}  
  
class Bus extends Vehicle {  
    Bus(String name) {  
        super(name);  
    }  
  
    double calculateRent(int days) {  
        return days * 2000;  
    }  
}
```

```

    }

    public class Main {

        public static void main(String[] args) {

            Vehicle[] v = new Vehicle[3];

            v[0] = new Car("Car");

            v[1] = new Bike("Bike");

            v[2] = new Bus("Bus");

            int days = 5;

            for (Vehicle x : v) {

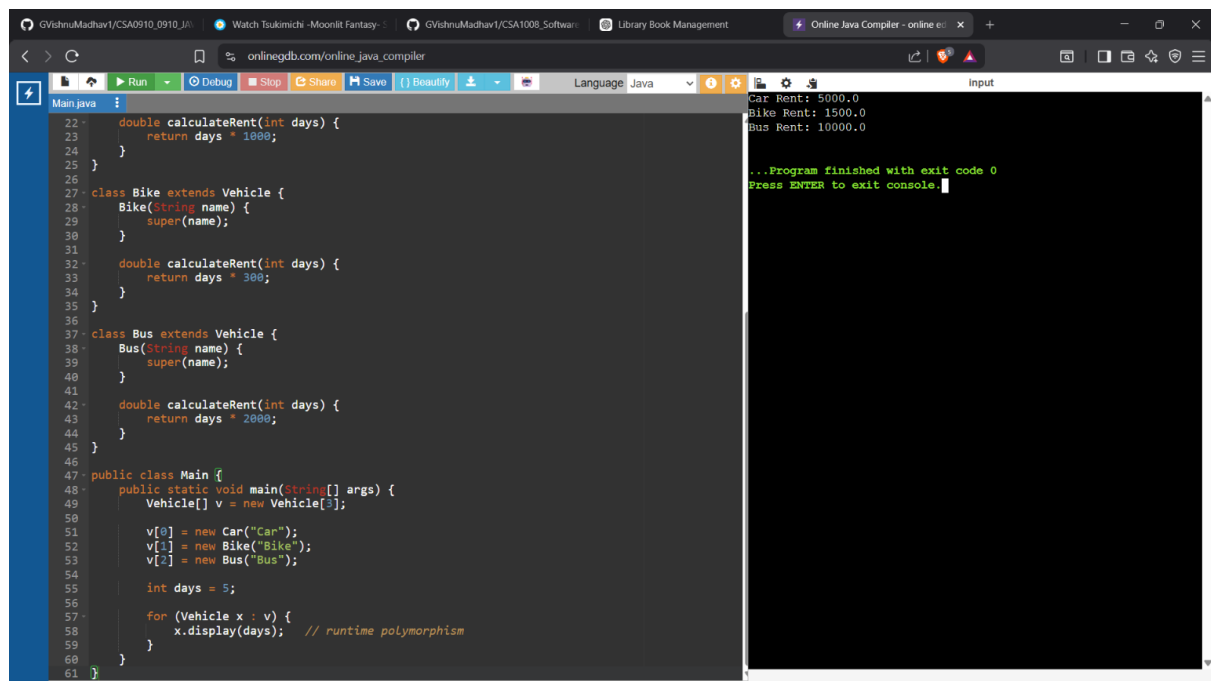
                x.display(days); // runtime polymorphism

            }

        }

    }
}

```



The screenshot shows a web browser window with an online Java compiler. The code is pasted into the editor, and the output is displayed in the console. The output shows the rent for each vehicle: Car Rent: 5000.0, Bike Rent: 1500.0, and Bus Rent: 10000.0. The program finished with exit code 0.

```

Main.java
22- double calculateRent(int days) {
23-     return days * 1000;
24- }
25-
26-
27- class Bike extends Vehicle {
28-     Bike(String name) {
29-         super(name);
30-     }
31-
32-     double calculateRent(int days) {
33-         return days * 300;
34-     }
35- }
36-
37- class Bus extends Vehicle {
38-     Bus(String name) {
39-         super(name);
40-     }
41-
42-     double calculateRent(int days) {
43-         return days * 2000;
44-     }
45- }
46-
47- public class Main {
48-     public static void main(String[] args) {
49-         Vehicle[] v = new Vehicle[3];
50-
51-         v[0] = new Car("Car");
52-         v[1] = new Bike("Bike");
53-         v[2] = new Bus("Bus");
54-
55-         int days = 5;
56-
57-         for (Vehicle x : v) {
58-             x.display(days); // runtime polymorphism
59-         }
60-     }
61- }

```

```

Car Rent: 5000.0
Bike Rent: 1500.0
Bus Rent: 10000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Bank Account System using Data Hiding

```

class BankAccount {

    private int accNo;

    private String name;

    private double balance;

```

```
BankAccount(int accNo, String name, double balance) {  
  
    this.accNo = accNo;  
  
    this.name = name;  
  
    this.balance = balance;  
  
}  
  
public void deposit(double amt) {  
  
    if (amt > 0) {  
  
        balance += amt;  
  
        System.out.println("Deposited");  
  
    } else {  
  
        System.out.println("Invalid Amount");  
  
    }  
  
}  
  
public void withdraw(double amt) {  
  
    if (amt > 0 && amt <= balance) {  
  
        balance -= amt;  
  
        System.out.println("Withdrawn");  
  
    } else {  
  
        System.out.println("Invalid Transaction");  
  
    }  
  
}  
  
public void showBalance() {  
  
    System.out.println("Balance: " + balance);  
  
}  
  
}
```

```

public class Main {

    public static void main(String[] args) {

        BankAccount b = new BankAccount(101, "Vishnu", 1000);

        b.deposit(500);

        b.withdraw(300);

        b.showBalance();

    }

}

```

```

Main.java
4 private double balance;
5
6 BankAccount(int accNo, String name, double balance) {
7     this.accNo = accNo;
8     this.name = name;
9     this.balance = balance;
10 }
11
12 public void deposit(double amt) {
13     if (amt > 0) {
14         balance += amt;
15         System.out.println("Deposited");
16     } else {
17         System.out.println("Invalid Amount");
18     }
19 }
20
21 public void withdraw(double amt) {
22     if (amt > 0 && amt <= balance) {
23         balance -= amt;
24         System.out.println("Withdrawn");
25     } else {
26         System.out.println("Invalid Transaction");
27     }
28 }
29
30 public void showBalance() {
31     System.out.println("Balance: " + balance);
32 }
33 }
34
35 public class Main {
36     public static void main(String[] args) {
37         BankAccount b = new BankAccount(101, "Vishnu", 1000);
38
39         b.deposit(500);
40         b.withdraw(300);
41         b.showBalance();
42     }
43 }

```

```

Deposited
Withdrawn
Balance: 1200.0

...Program finished with exit code 0
Press ENTER to exit console.

```

5. University Admission System using Method Overloading

```

class Admission {

    void calculateFee(int ugFee) {

        System.out.println("UG Fee: " + ugFee);

    }

    void calculateFee(int pgFee, String type) {

        System.out.println("PG Fee: " + pgFee);

    }

}

```

```

void calculateFee(int fee, double scholarship) {

    double finalFee = fee - (fee * scholarship);

    System.out.println("Scholarship Fee: " + finalFee);

}

}

public class Main {

    public static void main(String[] args) {

        Admission a = new Admission();

        a.calculateFee(50000);           // UG

        a.calculateFee(80000, "PG");     // PG

        a.calculateFee(50000, 0.5);      // Scholarship (50%)

    }

}

```

The screenshot shows an online Java compiler interface. The code is as follows:

```

1 class Admission {
2
3     void calculateFee(int ugFee) {
4         System.out.println("UG Fee: " + ugFee);
5     }
6
7     void calculateFee(int pgFee, String type) {
8         System.out.println("PG Fee: " + pgFee);
9     }
10
11    void calculateFee(int fee, double scholarship) {
12        double finalFee = fee - (fee * scholarship);
13        System.out.println("Scholarship Fee: " + finalFee);
14    }
15 }
16
17 public class Main {
18     public static void main(String[] args) {
19         Admission a = new Admission();
20
21         a.calculateFee(50000);           // UG
22         a.calculateFee(80000, "PG");     // PG
23         a.calculateFee(50000, 0.5);      // Scholarship (50%)
24     }
25 }

```

The output on the right side of the compiler shows:

```

UG Fee: 50000
PG Fee: 80000
Scholarship Fee: 25000.0

...Program finished with exit code 0
Press ENTER to exit console.

```

6. Shape Area Calculator using Abstract Class & Polymorphism

```

abstract class Shape {

```

```
    abstract double area();  
  
}  
  
class Circle extends Shape {  
  
    double r;  
  
    Circle(double r) { this.r = r; }  
  
    double area() {  
        return 3.14 * r * r;  
    }  
}
```

```
class Rectangle extends Shape {  
  
    double l, b;  
  
    Rectangle(double l, double b) {  
        this.l = l;  
        this.b = b;  
    }  
  
    double area() {  
        return l * b;  
    }  
}
```

```
class Triangle extends Shape {  
  
    double b, h;  
  
    Triangle(double b, double h) {  
        this.b = b;  
        this.h = h;  
    }  
}
```

```

    }

    double area() {

        return 0.5 * b * h;

    }

}

public class Main {

    public static void main(String[] args) {

        Shape[] s = new Shape[3];

        s[0] = new Circle(5);

        s[1] = new Rectangle(4, 6);

        s[2] = new Triangle(3, 8);

        for (Shape x : s) {

            System.out.println("Area: " + x.area()); // polymorphism

        }

    }

}

```

```

Main.java
11 }
12 }
13
14 class Rectangle extends Shape {
15     double l, b;
16     Rectangle(double l, double b) {
17         this.l = l;
18         this.b = b;
19     }
20
21     double area() {
22         return l * b;
23     }
24 }
25
26 class Triangle extends Shape {
27     double b, h;
28     Triangle(double b, double h) {
29         this.b = b;
30         this.h = h;
31     }
32
33     double area() {
34         return 0.5 * b * h;
35     }
36 }
37
38 public class Main {
39     public static void main(String[] args) {
40         Shape[] s = new Shape[3];
41
42         s[0] = new Circle(5);
43         s[1] = new Rectangle(4, 6);
44         s[2] = new Triangle(3, 8);
45
46         for (Shape x : s) {
47             System.out.println("Area: " + x.area()); // polymorphism
48         }
49     }
50 }

```

```

Area: 78.5
Area: 24.0
Area: 12.0

...Program finished with exit code 0
Press ENTER to exit console.

```

7. Hospital Management System using Interfaces

```
interface MedicalRecord {  
  
    void addRecord();  
  
    void displayRecord();  
  
}
```

```
class Patient implements MedicalRecord {
```

```
    int id;
```

```
    String name, disease;
```

```
    public void addRecord() {
```

```
        id = 1;
```

```
        name = "John";
```

```
        disease = "Fever";
```

```
    }
```

```
    public void displayRecord() {
```

```
        System.out.println("Patient ID: " + id);
```

```
        System.out.println("Name: " + name);
```

```
        System.out.println("Disease: " + disease);
```

```
    }
```

```
}
```

```
class Doctor implements MedicalRecord {
```

```
    int id;
```

```
    String name, specialization;
```

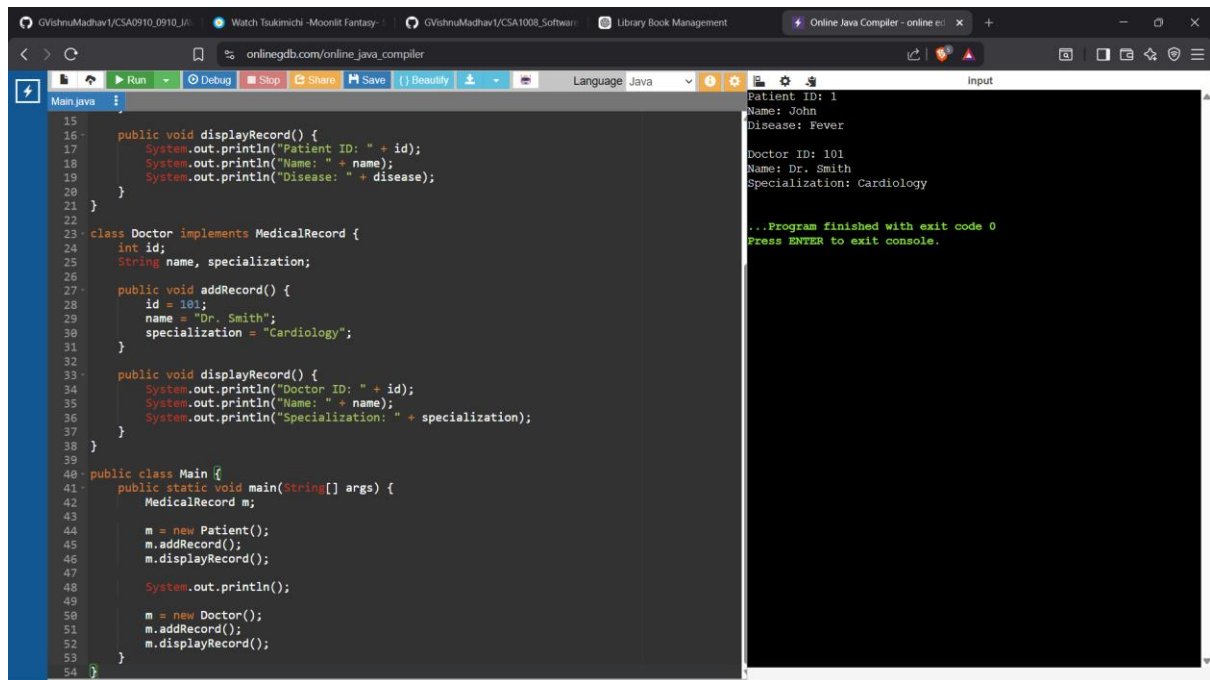
```
    public void addRecord() {
```

```
        id = 101;
```

```
    name = "Dr. Smith";  
  
    specialization = "Cardiology";  
  
}
```

```
public void displayRecord() {  
  
    System.out.println("Doctor ID: " + id);  
  
    System.out.println("Name: " + name);  
  
    System.out.println("Specialization: " + specialization);  
  
}  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        MedicalRecord m;  
  
  
  
        m = new Patient();  
  
        m.addRecord();  
  
        m.displayRecord();  
  
  
  
        System.out.println();  
  
  
  
        m = new Doctor();  
  
        m.addRecord();  
  
        m.displayRecord();  
  
    }  
}
```



The screenshot shows a web browser with an online Java compiler. The code on the left defines a `MedicalRecord` interface with `displayRecord()` and `addRecord()` methods. It implements these in a `Doctor` class and uses them in a `Main` class. The output on the right shows the execution results: Patient ID: 1, Name: John, Disease: Fever; and Doctor ID: 101, Name: Dr. Smith, Specialization: Cardiology. The program ends with an exit code of 0.

```
15 public void displayRecord() {
16     System.out.println("Patient ID: " + id);
17     System.out.println("Name: " + name);
18     System.out.println("Disease: " + disease);
19 }
20 }
21
22 class Doctor implements MedicalRecord {
23     int id;
24     String name, specialization;
25
26     public void addRecord() {
27         id = 101;
28         name = "Dr. Smith";
29         specialization = "Cardiology";
30     }
31
32     public void displayRecord() {
33         System.out.println("Doctor ID: " + id);
34         System.out.println("Name: " + name);
35         System.out.println("Specialization: " + specialization);
36     }
37 }
38
39 public class Main {
40     public static void main(String[] args) {
41         MedicalRecord m;
42
43         m = new Patient();
44         m.addRecord();
45         m.displayRecord();
46
47         System.out.println();
48
49         m = new Doctor();
50         m.addRecord();
51         m.displayRecord();
52     }
53 }
54
```

Patient ID: 1
Name: John
Disease: Fever

Doctor ID: 101
Name: Dr. Smith
Specialization: Cardiology

...Program finished with exit code 0
Press ENTER to exit console.

8. Online Shopping Order System using Packages

```
class Product {
```

```
    int id;
```

```
    String name;
```

```
    double price;
```

```
    Product(int id, String name, double price) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.price = price;
```

```
    }
```

```
}
```

```
class Order {
```

```
    double total = 0;
```

```
    void addProduct(Product p, int qty) {
```

```
        total += p.price * qty;
```

```
    }
```

```

void showBill() {

    System.out.println("Total Amount: " + total);

}

}

public class Main {

    public static void main(String[] args) {

        Product p1 = new Product(1, "Laptop", 50000);

        Product p2 = new Product(2, "Mouse", 500);

        Order o = new Order();

        o.addProduct(p1, 1);

        o.addProduct(p2, 2);

        o.showBill();

    }

}

```

The screenshot shows a web browser window with an online Java compiler. The code is pasted into the editor, and the 'Run' button has been clicked. The output console on the right displays the result of the program execution.

```

1 class Product {
2     int id;
3     String name;
4     double price;
5
6     Product(int id, String name, double price) {
7         this.id = id;
8         this.name = name;
9         this.price = price;
10    }
11 }
12
13 class Order {
14     double total = 0;
15
16     void addProduct(Product p, int qty) {
17         total += p.price * qty;
18     }
19
20     void showBill() {
21         System.out.println("Total Amount: " + total);
22     }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27
28         Product p1 = new Product(1, "Laptop", 50000);
29         Product p2 = new Product(2, "Mouse", 500);
30
31         Order o = new Order();
32
33         o.addProduct(p1, 1);
34         o.addProduct(p2, 2);
35
36         o.showBill();
37     }
38 }

```

Output:

```

Total Amount: 51000.0
...Program finished with exit code 0
Press ENTER to exit console.

```

9. Student Result Analyzer using Arrays of Objects

```
class Student {  
  
    int roll;  
  
    String name;  
  
    int m1, m2, m3;  
  
    Student(int r, String n, int a, int b, int c) {  
  
        roll = r;  
  
        name = n;  
  
        m1 = a;  
  
        m2 = b;  
  
        m3 = c;  
  
    }  
  
    int total() {  
  
        return m1 + m2 + m3;  
  
    }  
  
    double average() {  
  
        return total() / 3.0;  
  
    }  
  
    String grade() {  
  
        double avg = average();  
  
        if (avg >= 75) return "A";  
  
        else if (avg >= 60) return "B";  
  
        else if (avg >= 50) return "C";  
  
        else return "Fail";  
  
    }  
}
```

```

void display() {

    System.out.println(roll + " " + name +

        " Total: " + total() +

        " Avg: " + average() +

        " Grade: " + grade());

}

}

public class Main {

    public static void main(String[] args) {

        Student[] s = new Student[3];

        s[0] = new Student(1, "John", 80, 75, 90);
        s[1] = new Student(2, "Alice", 60, 65, 70);
        s[2] = new Student(3, "Bob", 50, 40, 45);

        int max = s[0].total();

        int topperIndex = 0;

        for (int i = 0; i < s.length; i++) {

            s[i].display();

            if (s[i].total() > max) {

                max = s[i].total();

                topperIndex = i;

            }

        }

    }

}

```

```

        System.out.println("Topper: " + s[topperIndex].name +
            " with " + max + " marks");
    }
}

```

```

// Student.java
class Student {
    int roll;
    String name;
    int[] marks;

    Student(int roll, String name, int[] marks) {
        this.roll = roll;
        this.name = name;
        this.marks = marks;
    }

    int total() {
        int sum = 0;
        for (int mark : marks) {
            sum += mark;
        }
        return sum;
    }

    double average() {
        return (double) total() / marks.length;
    }

    String grade() {
        double avg = average();
        if (avg >= 75) return "A";
        else if (avg >= 60) return "B";
        else if (avg >= 50) return "C";
        else return "Fail";
    }

    void display() {
        System.out.println(roll + " " + name +
            " Total: " + total() +
            " Avg: " + average() +
            " Grade: " + grade());
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        Student[] s = new Student[3];

        s[0] = new Student(1, "John", 80, 75, 90);
        s[1] = new Student(2, "Alice", 60, 65, 70);
        s[2] = new Student(3, "Bob", 50, 40, 45);

        int max = s[0].total();
        int topperIndex = 0;

        for (int i = 0; i < s.length; i++) {
            s[i].display();

            if (s[i].total() > max) {
                max = s[i].total();
                topperIndex = i;
            }
        }

        System.out.println("Topper: " + s[topperIndex].name +
            " with " + max + " marks");
    }
}

```

Output:

```

1 John Total: 245 Avg: 81.66666666666667 Grade: A
2 Alice Total: 195 Avg: 65.0 Grade: B
3 Bob Total: 135 Avg: 45.0 Grade: Fail
Topper: John with 245 marks

...Program finished with exit code 0
Press ENTER to exit console.

```

10. Smart Home Device Controller using Hierarchical Inheritance

```

class Device {
    String name;
    boolean status;

    Device(String name) {
        this.name = name;
        status = false;
    }

    void turnOn() {
        status = true;
        System.out.println(name + " ON");
    }
}

```

```
void turnOff() {  
    status = false;  
    System.out.println(name + " OFF");  
}
```

```
double power() {  
    return 0;  
}  
}
```

```
class Light extends Device {  
    Light(String name) {  
        super(name);  
    }  
}
```

```
double power() {  
    return status ? 10 : 0;  
}  
}
```

```
class Fan extends Device {  
    Fan(String name) {  
        super(name);  
    }  
}
```

```
double power() {  
    return status ? 50 : 0;  
}  
}
```

```
class AirConditioner extends Device {  
    AirConditioner(String name) {  
        super(name);  
    }  
}
```

```

double power() {
    return status ? 1500 : 0;
}
}

public class Main {

    public static void main(String[] args) {

        Device[] d = new Device[3];

        d[0] = new Light("Light");
        d[1] = new Fan("Fan");
        d[2] = new AirConditioner("AC");

        for (Device x : d) {
            x.turnOn(); // polymorphism

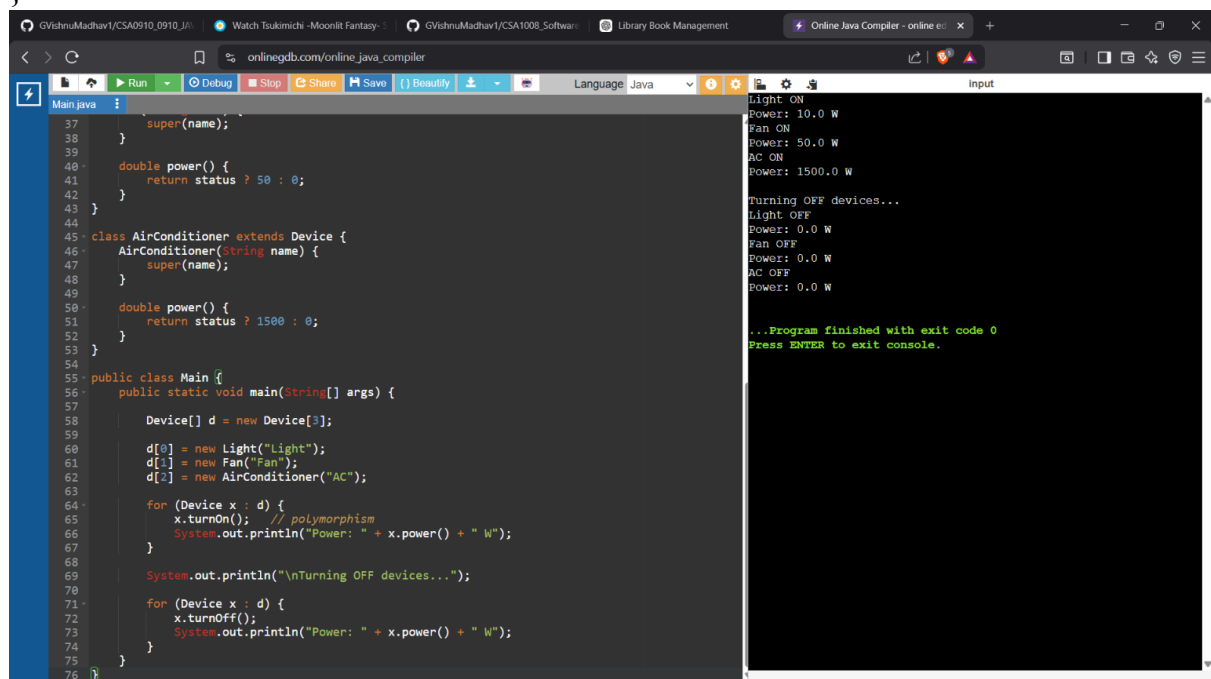
            System.out.println("Power: " + x.power() + " W");
        }

        System.out.println("\nTurning OFF devices...");

        for (Device x : d) {
            x.turnOff();

            System.out.println("Power: " + x.power() + " W");
        }
    }
}

```



The screenshot shows an online Java compiler interface. The left pane displays the source code, which includes a base class `Device` with a `power()` method, a subclass `AirConditioner` that extends `Device`, and a `Main` class that creates an array of `Device` objects (Light, Fan, AC), turns them on, prints their power, turns them off, and prints their power again. The right pane shows the output of the program, which matches the expected behavior: devices are turned on, their power is printed (10.0 W for Light, 50.0 W for Fan, 1500.0 W for AC), then they are turned off and their power is printed again (all 0.0 W). The program finishes with exit code 0.

```

Main.java
37     super(name);
38 }
39
40 double power() {
41     return status ? 50 : 0;
42 }
43 }
44
45 class AirConditioner extends Device {
46     AirConditioner(String name) {
47         super(name);
48     }
49
50     double power() {
51         return status ? 1500 : 0;
52     }
53 }
54
55 public class Main {
56     public static void main(String[] args) {
57
58         Device[] d = new Device[3];
59
60         d[0] = new Light("Light");
61         d[1] = new Fan("Fan");
62         d[2] = new AirConditioner("AC");
63
64         for (Device x : d) {
65             x.turnOn(); // polymorphism
66             System.out.println("Power: " + x.power() + " W");
67         }
68
69         System.out.println("\nTurning OFF devices...");
70
71         for (Device x : d) {
72             x.turnOff();
73             System.out.println("Power: " + x.power() + " W");
74         }
75     }
76 }

```

```

Light ON
Power: 10.0 W
Fan ON
Power: 50.0 W
AC ON
Power: 1500.0 W

Turning OFF devices...
Light OFF
Power: 0.0 W
Fan OFF
Power: 0.0 W
AC OFF
Power: 0.0 W

...Program finished with exit code 0
Press ENTER to exit console.

```